Reading and Writing Flash Memory

This tutorial demonstrates how to use the on-board Flash memory of the Portenta H7 to read and write data using the BlockDevice API provided by Mbed OS.

Author · Giampaolo Mancini, Sebastian Romero · Last revision · 11/03/2022

Overview

This tutorial demonstrates how to use the on-board Flash memory of the Portenta H7 to read and write data using the BlockDevice API provided by Mbed OS. As the internal memory is limited in size, we will also take a look at saving data to the QSPI Flash memory.

Goals

- Accessing the Portenta's internal Flash memory using Mbed's Flash In-Application Programming Interface
- Accessing the Portenta's QSPI Flash memory using Mbed's Flash In-Application Programming Interface
- Reading the memory's characteristics

Required Hardware and

Portenta H7 (ABX00042) or Portenta H7 Lite Connected (ABX00046)

USB-C® cable (either USB-A to USB-C® or USB-C® to USB-C®)

Arduino IDE 1.8.10+ or Arduino Pro IDE 0.0.4+ or Arduino CLI 0.13.0+

Mbed OS APIs for Flash Storage

Portenta's core is based on the Mbed operating system, allowing for Arduino APIs to be integrated using APIs exposed directly by Mbed OS.

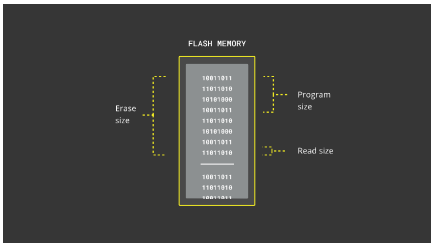
Mbed OS has a rich API for managing storage on different mediums, ranging from the small internal Flash memory of a microcontroller to external SecureDigital cards with large data storage space.

In this tutorial, you are going to save a value persistently inside the Flash memory. That allows to access that value even after rebooting the board. You will retrieve some information from a Flash block by using the FlashIAPBlockDevice API and create a block device object within the available space of the memory. In case of the internal memory, it corresponds to the space which is left after uploading a sketch to the board.

Be aware of the Flash r/w limits: Flash memories have a limited amount of read/write cycles. Typical Flash memories can perform about 10000 writes cycles to the same block before starting to "wear out" and begin to lose the ability to retain data. You can render your board useless with improper use of this example and described APIs.

Block Device Blocks

Blocks of Flash memory can be accessed through the block device APIs. They are byte addressable but operate in units of blocks. There are three types of blocks for the different block device operations: read blocks, erase blocks and program blocks. The recommended procedure for programming data is to first erase a block and then programming it in units of the program block size. The sizes of the erase, program and read blocks may not be the same, but they must be multiples of each another. Keep in mind that the state of an erased block is undefined until you program it with data.



Programming the Internal Flash

1. Create the Structure of the Program

Before we start it's important to keep the above mentioned **Flash r/w limits** in mind! Therefore this method should only be used for **once-in-a-while** read and write **operations**, such as reading a user setting in the `setup()`. It is not a good idea to use it for constantly updated values such as sensor data.

Having this in mind, it is time to create a sketch to program the Portenta. After creating new sketch and giving it a fitting name (in this case `FlashStorage.ino`), you need to create one more file to be used by the sketch, called `FlashIAPLimits.h`, that you will use to define some helper functions. This allows you to reuse the helper file later for other sketches.

2. The Helper Functions

Within the `FlashIAPLimits.h` file, you can start by including necessary libraries and defining the namespace.

```
// Ensures that this file
#pragma once

#include <Arduino.h>
#include <FlashIAP.h>
#include <FlashIAPBlockDe

using namespace mbed;
```

After that, you can create a struct which will later be used to save the

```
// An helper struct for F
struct FlashIAPLimits {
  size_t flash_size;
  uint32_t start_address;
  uint32_t available_size
};
```

The last part of the helper file consists of the `getFlashIAPLimits()` function used to calculate both the size of the Flash memory as well as the size and start address of the available memory.

This is done with Mbed's **FlashIAP** API. It finds the address of the first sector after the sketch stored in the microcontroller's ROM: `FLASHIAP_APP_ROM_END_ADDR` and uses the FlashIAP to calculate the Flash memory's size with `flash.get_flash_size()`. The other parameters can be determined using the same API.

```
// Get the actual start
// considering the spac
FlashIAPLimits getFlash
{
  // Alignment lambdas
  auto align_down = [](
    return (((val) / si
  };
  auto align_up = [](ui
    return (((val - 1)
  };

  size_t flash_size;
  uint32_t flash_start_
  uint32_t start_addres
  FlashIAP flash;

  auto result = flash.i
  if (result != 0)
    return { };

  // Find the start of
  int sector_size = fla
  start_address = aligr
  flash_start_address =
  flash_size = flash.ge

  result = flash.deinit
```

3. Reading & Writing Data

Going back to the `FlashStorage.ino` file, some more files need to be included in order to implement reading and writing to the Flash. The `FlashIAPBlockDevice.h` API will be used to create a block device in the empty part of the memory. Additionally, you can include the helper file `FlashIAPLimits.h` to have access to the address and size calculation function that you just created. You can also reference the `mbed` namespace for better readability.

```
#include <FlashIAPBlockDe
#include "FlashIAPLimits.

using namespace mbed;
```

The `setup()` function will first wait until a serial connection is established and then feed the random number generator, which is used later in this tutorial to write a random number in the Flash memory every time the device boots up.

```
void setup() {
  Serial.begin(115200);
  while (!Serial);

  Serial.println("FlashIA
  Serial.println("-------

  // Feed the random numb
  randomSeed(analogRead(0
```

Next the helper function, defined in the `FlashIAPLimits.h` file is called to calculate the memory properties, which are then used to create a block device using the `FlashIAPBlockDevice.h` library.

```
```

```
// Get limits of the the
auto [flashSize, startAd

Serial.print("Flash Size
Serial.print(flashSize /
Serial.println(" MB");
Serial.print("FlashIAP S
Serial.println(startAddr
Serial.print("FlashIAP S
Serial.print(iapSize / 1
Serial.println(" MB");

// Create a block device
FlashIAPBlockDevice bloc
```

Before using the block device, the first step is to initialize it using `blockDevice.init()`. Once initialized, it can provide the sizes of the blocks for programming the Flash. In terms of reading and writing Flash memory blocks, there is a distinction between the size of a readable block in bytes, a programmable block, which size is always a multiple of the read size, and an erasable block, which is always a multiple of a programmable block.

When reading and writing directly from and to the Flash memory, you need to always allocate a buffer with a multiple of the program block size. The amount of required program blocks can be determined by dividing the data size by the program block size. The final buffer size is equal to the amount of program blocks multiplied by the program block size.

```
// Initialize the Flash
blockDevice.init();

const auto eraseBlockSiz
const auto programBlockS

Serial.println("Block de
Serial.println("Readable
Serial.println("Programm
Serial.println("Erasable

String newMessage = "Ran

// Calculate the amount
// This has to be a mult
const auto messageSize =
const unsigned int requi
const unsigned int requi
const auto dataSize = re
char buffer[dataSize] {}
```

In the last part of the `setup()` function you can now use the block device to read and write data. First the buffer is used to read what was stored within the previous execution, then the memory gets erased and reprogrammed with the new content. At the end of the reading and writing process, the block device needs to be de-initialized again using `blockDevice.deinit()`.

```
// Read back what was st
Serial.println("Reading
blockDevice.read(buffer,
Serial.println(buffer);

// Erase a block startin
// to the block device s
blockDevice.erase(0, req

// Write an updated mess
Serial.println("Writing
Serial.println(newMessag
blockDevice.program(newM

// Deinitialize the devi
blockDevice.deinit();
Serial.println("Done.");
```

Finally the `loop()` function of this sketch will be left empty, considering that the Flash reading and writing process should be carried out as little as possible.

4. Upload the Sketch

Below is the complete sketch of this tutorial consisting of the main sketch and the `FlashIAPLimits.h` helper file, upload both of them to your Portenta H7 to try it out.

FlashIAPLimits.h

```
/**
Helper functions for ca
**/

// Ensures that this fi
#pragma once

#include <Arduino.h>
#include <FlashIAP.h>
#include <FlashIAPBlock

using namespace mbed;

// A helper struct for
struct FlashIAPLimits {
  size_t flash_size;
  uint32_t start_addres
  uint32_t available_si
};

// Get the actual start
// considering the spac
FlashIAPLimits getFlash
{
  // Alignment lambdas
  auto align_down = [](
    return (((val) / si
  };
```

FlashStorage.ino

```
#include <FlashIAPBlock
#include "FlashIAPLimit

using namespace mbed;

void setup() {
  Serial.begin(115200);
  while (!Serial);

  Serial.println("Flash
  Serial.println("-----

  // Feed the random nu
  randomSeed(analogRead

  // Get limits of the
  auto [flashSize, star

  Serial.print("Flash S
  Serial.print(flashSiz
  Serial.println(" MB")
  Serial.print("FlashIA
  Serial.println(startA
  Serial.print("FlashIA
  Serial.print(iapSize
  Serial.println(" MB")

  // Create a block dev
```

5. Results

After uploading the sketch open the Serial Monitor to start the Flash reading and writing process. The first time you start the script, the block device will be filled randomly. Now try to reset or disconnect the Portenta and reconnect it, you should see a message with the random number written to the Flash storage in the previous execution.

Note that the value written to the Flash storage will persist if the board is reset or disconnected. However, the Flash storage will be reprogrammed once a new sketch is uploaded to the Portenta and may overwrite the data stored in the Flash.

Programming the QSPI Flash

One issue with the internal Flash is that it is limited in size and the erase blocks are pretty large. This leaves very little space for your sketch and you may quickly run into issues with more complex applications. Therefore, you can use the external QSPI Flash which has plenty of space to store data. For that, the block device needs to be initialized differently, but the rest of the sketch remains the same. To initialize the device you can use the QSPIFBlockDevice class which is a block device driver for NOR-based QSPI Flash devices.

```
#define BLOCK_DEVICE_SIZ
#define PARTITION_TYPE 0

// Create a block device
QSPIFBlockDevice root(PD
MBRBlockDevice blockDevi

// Initialize the Flash
if(blockDevice.init() !=
  Serial.println("Partit
  blockDevice.deinit();
  // Allocate a FAT 32 p
  MBRBlockDevice::partit
  blockDevice.init();
```

While the QSPI block device memory can be used directly, it is better to use a partition table as the QSPI storage is also filled with other data, such as the Wi-Fi firmware. For that you use the MBRBlockDevice class and allocate a 8 KB partition, which can then be used to read and write data.

The full QSPI version of the sketch is as follows:

```
#include "QSPIFBlockDev
#include "MBRBlockDevic

using namespace mbed;

#define BLOCK_DEVICE_SI
#define PARTITION_TYPE

void setup() {
  Serial.begin(115200);
  while (!Serial);

  Serial.println("QSPI
  Serial.println("-----

  // Feed the random nu
  randomSeed(analogRead

  // Create a block dev
  QSPIFBlockDevice root
  MBRBlockDevice blockD

  // Initialize the Fla
  if(blockDevice.init()
    Serial.println("Par
    blockDevice.deinit(
    // Allocate a FAT 3
    MBRBlockDevice::par
```

Conclusion

We have learned how to use the

save custom data. It is not recommended to use the Flash of the microcontroller as the primary storage for data-intensive applications. It is better suited for read/write operations that are performed only once in a while such as storing and retrieving application configurations or persistent parameters.

Next Steps

Now that you know how to use block device to perform reading and writing Flash memory, you can look into the **next tutorial** on how to use the **TDBStore API** to create a **key value store** in the Flash memory.